

Day 1: Encapsulation Mechanics – A Beginner's Journey through the Digital Stack

1. The Strategic Foundation of Network Analysis

Mastering data flow is the "first principles" skill of any effective cybersecurity analyst. During my years managing network security for global operations at ExxonMobil and North American infrastructure at BP, I learned that you cannot defend what you do not understand at its most granular level. To understand data flow, look at digital forensics: an analyst uses the **Filesystem Hierarchy Standard (FHS)** to categorize data as **Shareable vs. Unshareable** and **Static vs. Variable**. Without this classification, you are just guessing.

In a network context, we apply the same logic. Just as the `/etc` directory is **Unshareable and Static** (host-specific and rarely changing), certain network headers are **Variable**—they change from hop to hop (like MAC addresses)—while the application payload remains **Static** and **Shareable** from end to end. This differentiation is what separates a "log-reader" from a "threat hunter." A log-reader sees a generic alert; a threat hunter knows exactly where a protocol downgrade occurs or where malicious data is hidden within a variable header to bypass a firewall.

2. Decoding the Models: OSI vs. TCP/IP via Analogy

The OSI (Open Systems Interconnection) and TCP/IP models are the maps we use to navigate the internet's infrastructure. While OSI is a seven-layer theoretical framework, the four-layer TCP/IP model is the practical reality I dealt with daily at Chevron when responding to incidents across the company's digital infrastructure.

To simplify these, we use the "**Filing and Shipping**" analogy:

- **Application/Presentation/Session Layers:** This is the creation of the document. Like an audit log in Google Cloud Search, you are defining the content and the context.
- **Transport Layer:** This is the socket connection. Using Python's `socket` library, this is where you define `AF_INET` (the address family) and `SOCK_STREAM` (TCP) to initiate the peer-to-peer conversation.
- **Network/Internet Layer:** This is the act of addressing the envelope. You are determining the source and destination IP addresses to ensure the package reaches the correct destination across the global web.
- **Data Link/Physical Layers:** This is the actual hardware and the wire. In Ubuntu, hardware is represented in the `/dev` directory, serving as the physical exit point for your bits.

OSI Layer	TCP/IP Layer	Primary Function
7. Application	Application	User-facing protocols (HTTP, DNS, SMTP)
6. Presentation	Application	Data representation and encryption

5. Session	Application	Inter-host communication management
4. Transport	Transport	End-to-end connections (TCP, UDP)
3. Network	Internet	Path determination and IP addressing
2. Data Link	Network Access	Physical addressing (MAC) and framing
1. Physical	Network Access	Hardware, cables, and bit transmission

3. The Journey Down the Stack: Encapsulation Mechanics

Encapsulation is the process of data traveling from a user's hands down to the physical wire. Each layer adds a "header" with instructions for that stage of the trip.

In modern protocols like TLS 1.3, we utilize a "**Hybrid Cryptosystem**" approach. The upper layers handle the **Handshake**, using public-key cryptography to establish a shared secret. The lower **Record** layer then uses fast symmetric cryptography for bulk data. A major efficiency in TLS 1.3 is the **1-RTT (Round Trip Time)** handshake: unlike older versions, the client sends its **cryptographic key shares in the very first flight (the first message)**, allowing the server to agree on a key immediately.

The step-by-step transformation:

- **Application to Transport:** Data is passed through an INET/STREAM "Socket."
 - *Unit: Data*
- **Transport to Network:** The layer segments the data and adds source/destination ports.
 - *Unit: Segment*
- **Network to Data Link:** The layer adds IP addresses for routing.
 - *Unit: Packet*
- **Data Link to Physical:** The packet is wrapped in a frame with MAC addresses and integrity checks, then converted to signals.
 - *Unit: Frame / Bits*

4. The Metadata Wrapper: Headers and Trailers

Headers and trailers are the "labels" on the digital envelope. For a SOC analyst, this metadata is the source of truth, answering the fundamental audit logging question: "**Who did what, where, and when?**"

Tools like Zeek extract this metadata into logs. For example, in a `conn.log`, the **Source IP** tells you "Who," while the **Service/Destination Port** tells you "What." Missing or forged headers are your primary indicators of a breach.

- **Headers:** These contain routing and security versioning.

- **Riley's Pro-Tip on Ossification:** When inspecting TLS 1.3 traffic, you may see the version header listed as **0x0303 (TLS 1.2)**. This is a deliberate "masquerade" to prevent **ossification**, where older middleboxes (firewalls/routers) drop packets because they don't recognize the new 1.3 versioning.
- **Trailers:** These follow the data, containing the **Frame Check Sequence (FCS)** or **Message Authentication Code (MAC)**. These ensure the packet was not tampered with or corrupted during its journey.

5. Exercise 1: Mapping the Web Request

Scenario: A user opens a browser to view an HTTPS webpage using TLS 1.3.

Task: Map this request to the TCP/IP model using the template below.

1. **Application:** [Identify the protocol and tool]
2. **Transport:** [Identify the connection protocol]
3. **Internet:** [Identify the addressing mechanism]
4. **Network Access:** [Identify the hardware/link layer]

6. Exercise 2: Packet Hunting with tcpdump

To hunt effectively, you must see data "in the wild." Professional analysts use `tcpdump` for command-line capture.

Task: Run the following command to capture five packets: `tcpdump -i any -c 5 -nn`

Observation: Locate the Source/Destination IP addresses and the ports. This is the raw metadata that populates your security logs.

7. Expert Solutions and Walkthroughs

Solution 1: Web Mapping

- **Application:** The browser uses **HTTP** to send a GET request.
- **Transport:** **TCP** manages the connection. In TLS 1.3, the client sends **cryptographic key shares** in the very first message to achieve a 1-RTT handshake.
- **Internet:** **IP** routes the request to the server's unique address.
- **Network Access:** Data is converted to an **Ethernet/Wi-Fi Frame** for the wire.

Solution 2: Packet Inspection

Here is a sample `tcpdump` output:

```
14:20:01.654321 IP 192.168.1.10.55234 > 93.184.216.34.443: Flags [S], seq 101, win 64240, length 0
```

- **Header Analysis:** The IP header ends after the destination IP, and the TCP header begins with the port.
 - **IP Header:** **192.168.1.10 > 93.184.216.34**

- **TCP Header: 55234 > 443**
- **Threat Hunter Insight (0-RTT vs 1-RTT):** If you see **0-RTT** data (session resumption), be aware of the **Replay Attack** risk. Because 0-RTT data is sent without a prior handshake, an attacker can capture and "replay" that packet. As an analyst, ensure your applications only allow "safe" requests like GETs in 0-RTT; state-changing requests (like a password update) in 0-RTT are a major security vulnerability.

Day 1 Takeaways:

1. **Encapsulation adds context:** Headers provide the "Who, What, Where, When" required for incident response.
2. **Models provide structure:** Understanding the FHS and TCP/IP models allows you to categorize data and predict where threats hide.
3. **Inspection provides truth:** Tools like `tcpdump` allow you to see the raw mechanics, bypassing logs that might be manipulated by an attacker.